Q1. Data communication using the OSI model

Seven layers: Application provides services such as HTTP; Presentation handles encoding, compression and encryption; Session manages dialogues; Transport provides process delivery; Network handles routing; Data Link handles local frames; Physical transmits signals.

Encapsulation: At the sender, each lower layer adds control information. Application data becomes a TCP segment, then an IP packet, then an Ethernet frame, and finally transmitted bits. Ethernet also adds a trailer.

Example: A browser sends an HTTP request. TCP adds ports and sequence numbers, IP adds source/destination IP addresses, and Ethernet adds local MAC addresses. For a remote server, the first destination MAC is the gateway's.

Decapsulation: The receiver checks and removes frame information, processes IP, and passes the payload to TCP. TCP reorders and acknowledges bytes, then delivers them to the browser/server application. Routers replace link frames at each hop.

Q2. OSI and TCP/IP models

Layer mapping: OSI Application + Presentation + Session → TCP/IP Application; Transport → Transport; Network → Internet; Data Link + Physical → Link. This uses the four-layer TCP/IP convention.

Similarities: Both divide networking into layers, use encapsulation, separate application/transport/routing functions, and support different physical networks.

Differences: OSI is a seven-layer reference framework with explicit service, interface and protocol boundaries. TCP/IP is a practical Internet protocol suite with broader layer boundaries and best-effort, connectionless IP.

OSI: Clear responsibilities help teaching, design and troubleshooting. Its strict boundaries do not always match real implementations.

TCP/IP: Open standards, interoperability and scalable routing support real networks. Session/presentation functions are less explicitly separated. Examples: HTTP/DNS, TCP/UDP, IP, Ethernet/Wi-Fi. Neither model alone guarantees security or reliability.

Q3. Troubleshooting using the seven OSI layers

Identify affected users/services and recent changes. For general connectivity failure, work upward and test one cause at a time.

1. Physical: Check power, cables, ports and wireless signal. Example: replace a damaged cable when the link light is off.

2. Data Link: Check VLANs, MAC/ARP tables and frame errors. Example: a wrong VLAN prevents reaching the gateway.

3. Network: Check IP address, mask, gateway and routes. Example: local access works but remote access fails because the gateway is wrong.

4. Transport: Check listeners, TCP/UDP ports and firewalls. Example: ping works but blocked port 443 prevents HTTPS.

5. Session: Check session state and expiry; expired tokens can cause repeated logouts. **6. Presentation:** Check encoding and TLS compatibility; a wrong clock can cause certificate errors.

7. Application: Check DNS, credentials, permissions and logs. Example: DNS failure prevents hostname access. Retest the original task after fixing the fault.

Q4. Sending a file to a different network

1. The application reads the file, manages the dialogue and applies encoding/encryption as needed. DNS resolves the destination name.

2. TCP establishes a connection and divides the byte stream into segments. Ports identify applications; sequence numbers, ACKs and retries provide reliable, ordered transfer.

3. IP adds the endpoint addresses. The sender identifies the remote subnet and selects its default gateway.

4. ARP obtains the gateway's MAC address. Ethernet encapsulates the IP packet; the physical layer sends signals. A switch forwards the frame using its MAC table.

5. Each router removes the incoming frame, selects an IP route, decrements TTL and creates a new outgoing frame. The final router addresses its frame to the receiver.

6. The receiver decapsulates the frame and packet. TCP checks and orders the bytes, and the application reconstructs the file. ACKs travel back. MAC addresses change per link; endpoint IPs normally remain the same without NAT.

Q5. Why networking uses layers

A layer groups related functions, uses the service below it and serves the layer above. A **protocol** governs peer communication; an **interface** governs interaction between adjacent layers.

Reduced complexity: Signaling, addressing, routing and reliable delivery can be studied and designed separately.

Interoperability: Standard protocols allow devices and software from different vendors to communicate.

Independent changes and reuse: Applications can use Ethernet or Wi-Fi through the same IP/transport services. Many applications reuse TCP rather than implementing reliability themselves.

Easier troubleshooting: Tests can isolate a cable fault, routing error or application problem. Clear responsibilities also simplify testing and maintenance.

Limitations: Extra headers and processing cause overhead; functions can be duplicated across layers. Strict separation may hide information that could improve performance.

Q6. TCP mechanisms

Service: TCP is connection-oriented and full-duplex. It provides a reliable, ordered byte stream, but does not preserve application message boundaries.

Establishment: SYN → SYN + ACK → ACK synchronizes sequence numbers and negotiates options.

Reliability: Byte sequence numbers, checksums, cumulative ACKs, adaptive retransmission timers and duplicate suppression recover missing/corrupt data. SACK can identify received blocks beyond a gap. Example: 500 bytes starting at 1001 are acknowledged by ACK 1501.

Flow control: The receiver advertises rwnd to avoid buffer overflow. **Congestion control:** The sender adjusts cwnd to limit network load. Outstanding data is limited by min(rwnd, cwnd).

Termination: Normally FIN → ACK → FIN → ACK closes the two directions independently. FIN consumes one sequence number; ACK and FIN can be combined. The active closer normally enters TIME-WAIT. RST aborts a connection.

Q7. TCP three-way handshake

Both endpoints must announce their initial sequence number and confirm receipt of the other's. Example with client sequence 1000 and server sequence 5000:

```
Client                        Server
   |--- SYN, seq=1000 ----------->|
   |<-- SYN+ACK, seq=5000, --------|
   |             ack=1001         |
   |--- ACK, seq=1001, ack=5001 -->|
```

Step 1: The client sends SYN and enters SYN-SENT. **Step 2:** The server acknowledges it, announces its own number and enters SYN-RECEIVED. **Step 3:** The client acknowledges the server; both can reach ESTABLISHED.

Why three steps? The third message confirms that the server's reply reached the client and helps reject stale connection attempts. Two messages would leave that confirmation missing.

SYN consumes one sequence number; a pure ACK does not. Lost SYN/SYN + ACK messages are retransmitted. The handshake creates connection state; it does not itself provide encryption or authentication.

Q8. TCP compared with UDP

Headers: TCP: minimum 20 bytes (up to 60 with options), including ports, sequence/ACK numbers, flags, window and checksum. UDP: 8 bytes, containing ports, length and checksum.

Reliability: TCP acknowledges, retransmits and delivers ordered bytes. UDP preserves datagram boundaries but provides no built-in delivery, ordering or duplicate guarantee.

Connection management: TCP uses a handshake and maintains state; FIN/ACK closes it. UDP has no protocol-level connection setup or close.

Control: TCP supplies receiver flow control and network congestion control. UDP applications or higher protocols must provide suitable controls themselves.

Overhead: UDP has a smaller header and no built-in handshake/retries. TCP adds state and recovery traffic. With 100 bytes of data and a 20-byte IPv4 header, totals are TCP 140 bytes and UDP 128 bytes, excluding link framing.

Uses: TCP: SSH, email, HTTP/1.1 and HTTP/2. UDP: many DNS queries, voice/video and games. QUIC adds reliable transport over UDP for HTTP/3.

Q9. TCP congestion control

In this **classical Reno** explanation, cwnd limits network load, rwnd protects the receiver, and ssthresh separates growth phases. MSS is the maximum segment payload size.

Slow Start: cwnd grows roughly exponentially per RTT as new data is acknowledged. Illustrative sequence: 1, 2, 4, 8 MSS. Actual initial windows and ACK patterns can differ.

Congestion Avoidance: At the threshold, growth becomes about 1 MSS per RTT. Per ACK, the approximate increase is MSS × MSS / cwnd.

Fast Retransmit: Three duplicate ACKs trigger retransmission of the likely missing segment before its timer expires.

Fast Recovery: Set ssthresh = max(flight size / 2, 2 MSS), then cwnd = ssthresh + 3 MSS. Further duplicate ACKs temporarily increase cwnd. In the single-loss case, an ACK covering recovery returns cwnd to ssthresh.

Timeout: Reduce ssthresh, reset cwnd to 1 MSS under the classical rule, back off the timer and restart slow start. Modern congestion-control algorithms may use different rules.

Q10. Go-Back-N and Selective Repeat ARQ

Both use sequence numbers, ACKs and retransmissions, and allow several frames in flight through a sliding window.

Go-Back-N (GBN): The receiver accepts only the next expected frame and discards out-of-order frames. ACKs are cumulative. On timeout, the sender repeats the oldest unacknowledged frame and all later outstanding frames. Receiver window = 1; buffering and control are simpler.

Selective Repeat (SR): The receiver buffers valid out-of-order frames and acknowledges them selectively. The sender retries only frames needing recovery, with individual status/timers. This saves bandwidth but needs more memory and bookkeeping.

Example: Send frames 1–4 and lose frame 2. Assuming successful ACKs/retries, GBN repeats 2, 3, 4: 7 total sends. SR repeats only 2: 5 sends.

Window limits: With m sequence bits, GBN sender window $\leq 2^m - 1$. Equal SR windows must be $\leq 2^{m-1}$ to avoid old/new frame ambiguity. Both recover errors; SR is usually more efficient on lossy, long-delay links.

Q1. Medium for factory buildings 2 km apart

Recommendation: Optical fiber, preferably single-mode fiber with optical transceivers rated for the required speed and 2 km distance.

Attenuation: Fiber supports kilometer-scale links with low loss. A typical twisted-pair Ethernet channel is limited to about 100 m, so a direct 2 km run is unsuitable.

Noise immunity: Light signals are immune to electromagnetic interference from motors and power cables. All-dielectric fiber also electrically isolates the buildings.

Bandwidth: Appropriate fiber and optics support high data rates and future upgrades. Capacity depends on the selected transceivers and distance rating.

Cost: Optics, termination and installation can cost more initially, but long reach avoids intermediate powered repeaters and improves reliability.

Arrangement: Switch A → optical fiber → Switch B. Allow for connector/splice losses and a safety margin in the optical link budget.

Q2. Multiplexing four voice/data channels

FDM: Assign channels A, B, C and D separate frequency bands. They transmit simultaneously, and receiver filters separate them.

Guard bands: Unused frequency gaps reduce adjacent-channel interference caused by imperfect filtering. For channel bandwidth B and three internal gaps g: total allocation = 4B + 3g. Example: B = 4 kHz and g = 1 kHz gives 19 kHz.

TDM: On a digital link, allocate repeating time slots A | B | C | D. Each source takes its turn at the full link rate; voice is digitized first if necessary.

Time separation: Synchronized slots prevent overlap; guard times may accommodate timing uncertainty. Guard time is a time gap, whereas a guard band is a frequency gap.

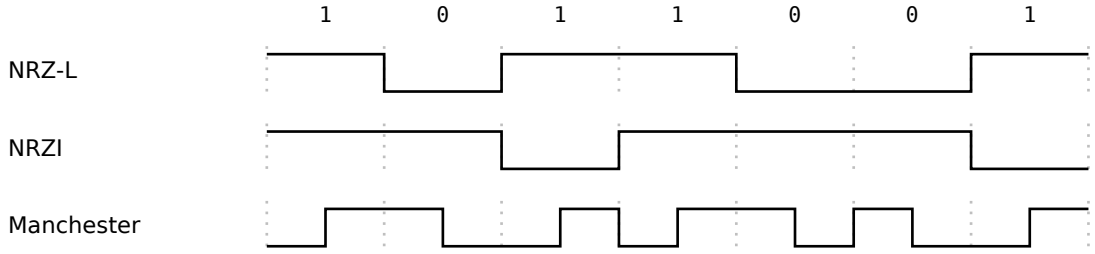
Four 64 kbit/s sources need at least 256 kbit/s of payload capacity plus overhead. Use FDM for simultaneous separate bands or TDM for coordinated digital channels. Fixed TDM slots can waste capacity when a source is idle.

Q3. NRZ, NRZI and Manchester for 1011001

NRZ-L: Use 1 = high (H), 0 = low (L). Levels: H L H H L L H. Simple, but long identical runs provide no timing transitions.

NRZI: Start at L; toggle at each 1 and keep the level for 0. Levels: H H L H H H L. Relative changes tolerate polarity inversion, but long zero runs lose clock transitions.

Manchester: Use 1 = LH and 0 = HL. Half-bit pairs: LH | HL | LH | LH | HL | HL | LH. A midpoint transition enables clock recovery, at the cost of greater signaling bandwidth.



Conventions are stated explicitly because opposite polarity conventions are also used. Time increases to the right; vertical dotted lines mark bit boundaries.

Q4. Pure ALOHA and Slotted ALOHA

Pure ALOHA: Send whenever data is ready. Overlapping packets collide; retransmit after random backoff. For packet duration T, the vulnerable period is 2T.

Slotted ALOHA: Transmit only at synchronized slot boundaries. The vulnerable period is T, reducing opportunities for collision.

Ideal throughput: With offered attempt load G (including retries), Pure: $S = G\, e^{-2G}$; Slotted: $S = G\, e^{-G}$.

Pure peaks at G = 0.5: $S = 1/(2e) \approx$ **18.4%**. Slotted peaks at G = 1: $S = 1/e \approx$ **36.8%**.

Recommendation: Slotted ALOHA is more efficient if sensors can maintain common timing; its ideal maximum throughput is twice that of Pure ALOHA.

Assumptions: Equal packet lengths, synchronized slots, independent Poisson attempts and loss of overlapping packets. Real timing errors, capture effects and synchronization overhead can change performance.

Q5. Ethernet collision detection and Wi-Fi avoidance

Traditional shared Ethernet: CSMA/CD. A station senses the cable, transmits if idle and monitors while transmitting. On collision, it stops, signals the collision and retries after binary exponential backoff.

Why detection works: A wired station can detect the electrical effect of simultaneous transmission. Example: computers sharing an old half-duplex Ethernet hub. Modern full-duplex switched Ethernet does not use CSMA/CD.

Wi-Fi: CSMA/CA. A radio's strong transmitted signal masks weak incoming signals, so it cannot reliably detect collisions while sending. Hidden terminals may hear the access point without hearing each other.

Avoidance: Wait for idle medium, apply interframe waiting and random backoff, freeze the counter while busy, then transmit. A missing unicast ACK leads to retry. Optional RTS/CTS helps with hidden terminals.

Scenario: Classroom laptops sharing an access point use collision avoidance. Carrier sensing and backoff reduce collisions but cannot eliminate them.

Q6. CRC for data 1101011011 and generator 10011

The generator has degree 4, so append four zeros. Divide using modulo-2 arithmetic: subtraction is XOR, with no carries or borrows.

```
Data + zeros: 11010110110000
XOR 10011000000000 -> 01001110110000
XOR 01001100000000 -> 00000010110000
XOR 00000010011000 -> 00000000101000
XOR 00000000100110 -> 00000000001110
```

CRC remainder = 1110.
Transmitted codeword = 11010110111110.

Receiver: Divide the entire received codeword by 10011 without appending zeros. Remainder 0000 means no error was detected; a nonzero remainder indicates corruption.

Example: changing the last bit gives 11010110111111, with remainder 0001. CRC detects errors but does not correct them. Error patterns divisible by the generator can remain undetected.

Q7. Stop-and-Wait utilization

Given: One-way propagation T_p = 20 ms; frame transmission T_t = 5 ms. Assume negligible ACK transmission/processing, immediate ACKs and no errors.

Each cycle includes frame transmission plus outward and return propagation:
Cycle = $T_t + 2T_p = 5 + 40 = 45$ ms.

Utilization $U = T_t / (T_t + 2T_p) = 5/45 = 1/9 \approx 11.1\%$.

Timing: transmit during 0–5 ms; the final bit reaches the receiver at 25 ms; the ACK returns at 45 ms. The sender transmits for 5 ms and waits for about 40 ms.

Effect of more delay: If T_p doubles to 40 ms, $U = 5/(5 + 80) \approx 5.88\%$. Thus larger propagation delay lowers utilization when frame time is fixed.

Pipelining improves use of the link. Under these ideal assumptions, about $(T_t + 2T_p)/T_t = 9$ outstanding frames can keep the sender busy.

Q8. 15 packets, window 5: GBN versus SR

Loss rule: First attempts for P4, P8 and P12 are lost. All retransmissions succeed; ACKs are never lost. Count data transmissions only.

GBN assumptions: Conventional cumulative ACKs, receive window 1, timeout recovery and no fast retransmit. ACKs allow the sender window to fill before timeout. Complete each retry group and its ACKs before sending new packets.

GBN trace: Send originals P1–P8; P4/P8 are lost and P5–P7 are discarded. Retry P4–P8 (5 sends). Send originals P9–P15; P12 is lost and P13–P15 are discarded. Retry P12–P15 (4 sends).

GBN total = 15 originals + 9 retries = 24 sends.

SR: Buffer valid out-of-order packets and retry only P4, P8 and P12. **Total = 15 + 3 = 18 sends.** This saves 6 sends (25% of the GBN total), using more buffering and control state.

The exact GBN count depends on ACK/timeout timing; 24 applies to the explicit schedule above. Losing every fourth attempt, including retries, would be a different loss rule.

Q9. Four equal subnets from 192.168.10.0/24

Borrow 2 host bits because $2^2 = 4$ subnets. New prefix: **/26**. New mask: **255.255.255.192**. Six host bits remain.

Each subnet: $2^6 = 64$ total addresses and $64 - 2 =$ **62 usable hosts**. Block increment = $256 - 192 = 64$.

Network (/26)	Usable host range	Broadcast
192.168.10.0	.1 - .62	.63
192.168.10.64	.65 - .126	.127
192.168.10.128	.129 - .190	.191
192.168.10.192	.193 - .254	.255

All abbreviated addresses in the table begin with **192.168.10**. For example, .65 means 192.168.10.65.

All-zero host bits identify the network; all-one host bits identify the broadcast. First usable = network + 1; last usable = broadcast − 1.

Check: four blocks × 64 addresses = 256, covering the original /24 without gaps or overlap.

Q10. Smallest IPv4 subnet for 50 usable hosts

For an ordinary IPv4 LAN, choose the smallest number of host bits h satisfying $\mathbf{2^h - 2 \geq 50}$. The two reserved addresses are the network and broadcast.

Try h = 5: $2^5 - 2 = 30$ usable hosts. Prefix /27 is too small.

Try h = 6: $2^6 - 2 = 62$ usable hosts. This meets the requirement.

Required prefix = 32 − 6 = /26.
Subnet mask = 255.255.255.192.

Example: 192.168.10.0/26 has network 192.168.10.0, usable range 192.168.10.1–192.168.10.62 and broadcast 192.168.10.63.

A /27 supplies only 30 hosts; a /25 supplies 126 but allocates more addresses than necessary. Thus /26 is the smallest suitable block. After 50 allocations, 12 usable addresses remain; gateways and other infrastructure also use this pool.